

JavaScript Basics for Document Q&A

Version: 1.0

Audience: new developers building browser or Node.js apps

Purpose: provide a compact, searchable document for testing a RAG chatbot.

1. What JavaScript Is

JavaScript is a programming language used to make web pages interactive and to build server applications with Node.js. In the browser, JavaScript can respond to clicks, update page content, validate forms, and call APIs. In Node.js, JavaScript can read files, run web servers, access databases, and automate tasks.

JavaScript is single-threaded at the language level, but it can handle many waiting tasks with an event loop. The event loop lets JavaScript continue running while timers, network requests, and file operations are waiting for results.

2. Variables and Values

Use `const` by default when a variable should not be reassigned. Use `let` when the value needs to change later. Avoid `var` in modern code because it has function scope and can create confusing behavior.

Example:

```
“js
const userName = "Mina";
let loginCount = 1;
loginCount = loginCount + 1;
“
```

Common JavaScript value types include strings, numbers, booleans, null, undefined, arrays, objects, and functions. A string stores text. A number stores numeric values. A boolean is either true or false. Null means an intentional empty value. Undefined usually means a value has not been assigned yet.

3. Functions

A function groups reusable logic. Functions can receive input through parameters and return output with the `return` keyword.

Example:

```
“js
function greet(name) {
  return Hello, ${name};
}
```

```
const message = greet("Samir");  
""
```

Arrow functions are a shorter function syntax often used for callbacks:

```
""js  
const double = (number) => number * 2;  
""
```

When a function does not explicitly return a value, JavaScript returns undefined.

4. Arrays and Objects

An array stores an ordered list of values. Use array methods like map, filter, find, and reduce to transform or search lists.

Example:

```
""js  
const scores = [8, 12, 15];  
const passed = scores.filter((score) => score >= 10);  
""
```

An object stores named properties. Objects are useful for representing things like users, products, messages, or settings.

Example:

```
""js  
const user = {  
  id: 42,  
  name: "Leila",  
  role: "admin"  
};  
""
```

Use dot notation when the property name is known, such as user.name. Use bracket notation when the property name is stored in a variable, such as user[fieldName].

5. Control Flow

Use if, else if, and else when code should run only under certain conditions. Use switch when comparing one value against several known cases. Use loops when code must repeat.

Example:

```
""js  
if (score >= 10) {
```

```
    status = "passed";
  } else {
    status = "needs review";
  }
}
```

Prefer readable conditions. If a condition becomes too long, store part of it in a well-named variable.

6. Async JavaScript

Asynchronous code handles work that finishes later, such as calling an API or reading a file. A Promise represents a future result. Use `async` and `await` to write Promise-based code in a readable style.

Example:

```
“js
async function loadUser(userId) {
  const response = await fetch(/api/users/${userId});
  const user = await response.json();
  return user;
}
“
```

Use `try` and `catch` to handle errors in `async` functions:

```
“js
try {
  const user = await loadUser(42);
  console.log(user.name);
} catch (error) {
  console.error("Could not load user", error);
}
“
```

7. DOM Events

The Document Object Model, or DOM, represents the page as objects that JavaScript can read and change. Use event listeners to respond to user actions.

Example:

```
“js
const button = document.querySelector("#save-button");

button.addEventListener("click", () => {
  console.log("Saved");
});
“
```

```
});  
""
```

Common DOM events include click, submit, input, change, keydown, and load. For forms, listen to the submit event and call `event.preventDefault()` when JavaScript should handle the submission without a full page reload.

8. Node.js and npm

Node.js lets JavaScript run outside the browser. Express.js is a popular Node.js framework for building APIs. npm is the package manager used to install libraries and run scripts.

Common npm commands:

- `npm install` installs project dependencies.
- `npm run dev` runs the development script defined in `package.json`.
- `npm run build` creates a production build when the project supports it.

In Node.js projects, environment variables are often stored in a `.env` file during development. API keys should not be committed to git.

9. Debugging Checklist

When JavaScript code fails, check the error message first. Then confirm the line number, inspect variable values, and simplify the failing code path. In browser apps, use the developer console and the network tab. In Node.js apps, use console logs, stack traces, and request logs.

Useful questions:

- Is the variable defined before it is used?
- Is the function receiving the input shape it expects?
- Did an async function need `await`?
- Did the API return an error response?
- Is the environment variable loaded?

10. Mini Project: Document Q&A Chatbot

The chatbot project has two main parts: ingestion and question answering. Ingestion loads a PDF, extracts text, splits text into chunks, embeds those chunks, and stores them in a vector store. Question answering embeds the user question, retrieves relevant chunks, sends those chunks to an LLM, and returns an answer with sources.

For a first version, use one sample PDF and an in-memory vector store. This makes the app easier to understand before adding persistent storage. Later, replace the in-memory store with Chroma, Pinecone, Postgres pgvector, or another persistent vector database.

The backend should expose endpoints for health checks, indexing the sample document, uploading a document, checking index status, and asking a question. The frontend should provide a document control area, an upload control, and a chat interface.

11. Glossary

Chunk: a smaller section of a document used for retrieval.

Embedding: a numeric representation of text meaning.

Vector store: a database or in-memory structure that searches embeddings by similarity.

Retriever: a component that finds relevant chunks for a query.

RAG: retrieval-augmented generation, a pattern where an LLM answers using retrieved context.

Prompt: the instruction and context sent to an LLM.

Hallucination: an answer that sounds confident but is not supported by the provided source material.

12. Recommended First Questions

Ask these questions to test the chatbot:

- What is the difference between const, let, and var?
- How does async and await help with API calls?
- What are the main steps in a Document Q&A chatbot?
- Why should a first version use an in-memory vector store?
- What should I check when JavaScript code fails?